Data Structures Laboratory Manual (Turbo C++)

Updated Manual (Programs 6 to 15) with 7-Step Algorithms for Programs 9 to 15

Program 6: Infix to Postfix using Stack

AIM

To convert an infix expression to a postfix expression using stack in Turbo C++.

ALGORITHM (9 STEPS)

1. Start the program and initialize an empty character stack s and string post = "".
2. Scan the infix expression from left to right character by character.
3. If the character is an operand (letter/digit), append it directly to post.
4. If the character is '(', push it onto stack s.
5. If the character is ')', pop from s and append to post until '(' is popped; discard '('.
6. If the character is an operator, check its precedence and associativity against s.top().
7. For +, -, *, /, pop while prec(s.top()) >= prec(ch); for ^, pop while prec(s.top()) > prec(ch).
8. Push the current operator onto stack s.
9. Pop and append all remaining operators from s to post, display post, and stop.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>
#include <ctype.h>

char st[50];
int top = -1;

void push(char c) { st[++top] = c; }
char pop() { return st[top--]; }
char peek() { return st[top]; }
int isEmpty() { return top == -1; }

int pr(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;
}

void main() {
    char in[] = "a+b*(c^d-e)", post[50];
    int k = 0;
    clrscr();
    for (int i = 0; in[i]; i++) {
        char c = in[i];
        if (isalnum(c)) post[k++] = c;
        else if (c == '(') push(c);
        else if (c == ')') {
            while (!isEmpty() && peek() != '(')
                post[k++] = pop();
            if (!isEmpty()) pop();
        } else {
            while (!isEmpty() && (c == '^' ?
                   pr(peek()) > pr(c) :
                   pr(peek()) >= pr(c))) {
                post[k++] = pop();
            }
            push(c);
        }
    }
    while (!isEmpty()) post[k++] = pop();
    post[k] = '\0';
    cout << "Infix  : " << in << "\n";
    cout << "Postfix: " << post << "\n";
    getch();
}
```

OUTPUT

```
Infix  : a+b*(c^d-e)
Postfix: abcd^e-*+
```

Result: Successfully converted infix expression to postfix expression in Turbo C++.

Program 7: Binary Tree Traversal

AIM

To implement Preorder, Inorder, and Postorder traversals of a Binary Tree in Turbo C++.

ALGORITHM (9 STEPS)

1. Start the program and define struct Node containing d, pointer l, and pointer r.
2. Dynamically allocate root and child nodes using new and link them to build the tree.
3. For Inorder Traversal, check if root is not null; recursively traverse the left subtree.
4. Print the current root data, then recursively traverse the right subtree.
5. For Preorder Traversal, check if root is not null; print the current root data.
6. Recursively traverse the left subtree, then recursively traverse the right subtree.
7. For Postorder Traversal, check if root is not null; recursively traverse the left subtree.
8. Recursively traverse the right subtree, then print the current root data.
9. Call all three traversal functions in main(), print the sequences, and stop.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

struct Node {
    int d;
    Node *l, *r;
};

Node* create(int v) {
    Node* n = new Node;
    n->d = v; n->l = NULL; n->r = NULL;
    return n;
}

void inorder(Node* r) {
    if (!r) return;
    inorder(r->l);
    cout << r->d << " ";
    inorder(r->r);
}

void preorder(Node* r) {
    if (!r) return;
    cout << r->d << " ";
    preorder(r->l);
    preorder(r->r);
}

void postorder(Node* r) {
    if (!r) return;
    postorder(r->l);
    postorder(r->r);
    cout << r->d << " ";
}

void main() {
    clrscr();
    Node* root = create(1);
    root->l = create(2);
    root->r = create(3);
    root->l->l = create(4);
    cout << "Preorder : "; preorder(root);
    cout << "\nInorder  : "; inorder(root);
    cout << "\nPostorder: "; postorder(root);
    cout << "\n";
    getch();
}
```

OUTPUT

```
Preorder : 1 2 4 3
Inorder  : 4 2 1 3
Postorder: 4 2 3 1
```

Result: Binary tree traversals executed successfully in Turbo C++.

Program 8: Binary Search Tree Operations

AIM

To implement Insertion, Searching, Deletion, and Inorder Display in a BST in Turbo C++.

ALGORITHM (9 STEPS)

1. Start the program and define struct Node with integer k, l, and r child pointers.
2. To insert a key: if root is null, create and return a new node with the key.
3. If key is smaller than root key, insert left; if greater, insert right; ignore duplicates.
4. To search a key: recursively move left if key is smaller, or right if key is greater, until matched or null.
5. To delete a key: traverse the BST recursively to locate the node to be removed.
6. If the node has 0 or 1 child, replace the node with its non-null child and free/delete memory.
7. If the node has 2 children, find its inorder successor (minimum key in right subtree).
8. Copy the successor's key into the current node and delete the successor from the right subtree.
9. Display the updated tree using inorder traversal, verify search/delete results, and stop.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

struct Node {
    int k;
    Node *l, *r;
};

Node* ins(Node* r, int v) {
    if (!r) {
        Node* n = new Node;
        n->k = v; n->l = NULL; n->r = NULL;
        return n;
    }
    if (v < r->k) r->l = ins(r->l, v);
    else if (v > r->k) r->r = ins(r->r, v);
    return r;
}

Node* minN(Node* n) {
    while (n && n->l) n = n->l;
    return n;
}

Node* del(Node* r, int v) {
    if (!r) return NULL;
    if (v < r->k) r->l = del(r->l, v);
    else if (v > r->k) r->r = del(r->r, v);
    else {
        if (!r->l) {
            Node* t = r->r; delete r; return t;
        }
        if (!r->r) {
            Node* t = r->l; delete r; return t;
        }
        Node* t = minN(r->r);
        r->k = t->k;
        r->r = del(r->r, t->k);
    }
    return r;
}

int search(Node* r, int v) {
    if (!r) return 0;
    if (r->k == v) return 1;
    if (v < r->k) return search(r->l, v);
    return search(r->r, v);
}

void disp(Node* r) {
    if (!r) return;
    disp(r->l);
    cout << r->k << " ";
    disp(r->r);
}

void main() {
    clrscr();
    Node* root = NULL;
    int a[] = {50, 30, 70, 20, 40};
    for (int i = 0; i < 5; i++)
        root = ins(root, a[i]);
    cout << "BST     : "; disp(root); cout << "\n";
```

```
    cout << "Find 30 : "
         << (search(root, 30) ? "Yes" : "No")
         << "\n";
    root = del(root, 30);
    cout << "AfterDel: "; disp(root); cout << "\n";
    getch();
}
```

OUTPUT

```
BST     : 20 30 40 50 70
Find 30 : Yes
AfterDel: 20 40 50 70
```

Result: BST operations implemented and executed successfully in Turbo C++.

Program 9: Heap Sort

AIM

To implement Heap Creation and sort an array using Heap Sort in Turbo C++.

ALGORITHM (7 STEPS)

1. Start the program and initialize the unsorted array a[] of size n.
2. Define heapify(a, n, i) with mx = i, left child = 2*i + 1, and right child = 2*i + 2.
3. If left or right child is greater than a[mx], update mx accordingly.
4. If mx != i, swap a[i] with a[mx] and recursively call heapify(a, n, mx).
5. In heapSort(), build max-heap by calling heapify from index i = n/2 - 1 down to 0.
6. Iterate from i = n - 1 down to 1: swap a[0] with a[i] and call heapify(a, i, 0).
7. Print the final sorted array, call getch(), and stop the program.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

void swap(int &x, int &y) {
    int t = x; x = y; y = t;
}

void heapify(int a[], int n, int i) {
    int mx = i, l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && a[l] > a[mx]) mx = l;
    if (r < n && a[r] > a[mx]) mx = r;
    if (mx != i) {
        swap(a[i], a[mx]);
        heapify(a, n, mx);
    }
}

void heapSort(int a[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(a, n, i);
    for (int j = n - 1; j > 0; j--) {
        swap(a[0], a[j]);
        heapify(a, j, 0);
    }
}

void main() {
    clrscr();
    int a[] = {12, 11, 13, 5, 6, 7}, n = 6;
    heapSort(a, n);
    cout << "Sorted: ";
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << "\n";
    getch();
}
```

OUTPUT

```
Sorted: 5 6 7 11 12 13
```

Result: Heap Sort algorithm executed successfully in Turbo C++.

Program 10: Graph Traversal – BFS

AIM

To implement Breadth First Search (BFS) graph traversal using Adjacency Matrix in Turbo C++.

ALGORITHM (7 STEPS)

1. Start the program and represent the graph using adjacency matrix g[V][V].
2. Initialize visited array vis[V] to 0 and create a queue q with front = 0, rear = -1.
3. Set start vertex, mark vis[start] = 1, and enqueue start into q.
4. Loop while queue is not empty (f <= r), dequeue front vertex u and print it.
5. Scan all vertices v from 0 to V - 1 to find adjacent vertices of u.
6. If edge g[u][v] == 1 and vis[v] == 0, mark vis[v] = 1 and enqueue v into q.
7. Repeat until queue becomes empty, print newline, and stop the program.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

const int V = 5;

void bfs(int g[V][V], int start) {
    int vis[V], q[20], f = 0, r = -1;
    for (int i = 0; i < V; i++) vis[i] = 0;
    vis[start] = 1;
    q[++r] = start;
    cout << "BFS: ";
    while (f <= r) {
        int u = q[f++];
        cout << u << " ";
        for (int v = 0; v < V; v++) {
            if (g[u][v] == 1 && vis[v] == 0) {
                vis[v] = 1;
                q[++r] = v;
            }
        }
    }
    cout << "\n";
}

void main() {
    clrscr();
    int g[V][V] = {
        {0, 1, 1, 0, 0},
        {1, 0, 0, 1, 1},
        {1, 0, 0, 0, 1},
        {0, 1, 0, 0, 0},
        {0, 1, 1, 0, 0}
    };
    bfs(g, 0);
    getch();
}
```

OUTPUT

```
BFS: 0 1 2 3 4
```

Result: Breadth First Search completed successfully in Turbo C++.

Program 11: Graph Traversal – DFS

AIM

To implement Depth First Search (DFS) graph traversal using Adjacency List in Turbo C++.

ALGORITHM (7 STEPS)

1. Start the program and initialize adjacency list array adj[V] with NULL.
2. Add edges by dynamically allocating EdgeNode and linking to head of adj[u].
3. Initialize visited array vis[V] with 0 to track visited vertices.
4. In dfsUtil(u): mark vis[u] = 1 and print current vertex u.
5. Traverse adjacency list of u; if neighbor p->v is unvisited, call dfsUtil(p->v).
6. In dfs(): loop all vertices 0 to V - 1 and call dfsUtil for any unvisited node.
7. Display the traversal sequence, call getch(), and stop the program.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

const int V = 4;

struct EdgeNode {
    int v;
    EdgeNode* next;
};

EdgeNode* adj[V];
int vis[V];

void addEdge(int u, int v) {
    EdgeNode* n = new EdgeNode;
    n->v = v;
    n->next = adj[u];
    adj[u] = n;
}

void dfsUtil(int u) {
    vis[u] = 1;
    cout << u << " ";
    for (EdgeNode* p = adj[u]; p; p = p->next) {
        if (!vis[p->v]) dfsUtil(p->v);
    }
}

void dfs() {
    for (int i = 0; i < V; i++) vis[i] = 0;
    cout << "DFS: ";
    for (int j = 0; j < V; j++) {
        if (!vis[j]) dfsUtil(j);
    }
    cout << "\n";
}

void main() {
    clrscr();
    for (int i = 0; i < V; i++) adj[i] = NULL;
    addEdge(0, 1); addEdge(0, 2);
    addEdge(1, 3); addEdge(2, 3);
    dfs();
    getch();
}
```

OUTPUT

```
DFS: 0 2 3 1
```

Result: Depth First Search completed successfully in Turbo C++.

Program 12: String Pattern Matching

AIM

To implement Brute Force and KMP pattern matching algorithms in Turbo C++.

ALGORITHM (7 STEPS)

1. Start the program and read text string txt and pattern string pat.
2. If pattern is empty (m == 0) or m > n, terminate search safely.
3. In Brute Force: compare pat with txt at each starting index i from 0 to n - m.
4. If all m characters match, print index i as match position.
5. In KMP preprocessing: compute lps[] array using longest prefix-suffix logic.
6. In KMP search: match txt[i] and pat[j]; on mismatch, shift j using lps[j - 1] without backtracking i.
7. Print all match positions for both algorithms, and stop the program.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>
#include <string.h>

void brute(char t[], char p[]) {
    int n = strlen(t), m = strlen(p);
    if (m == 0 || n < m) return;
    cout << "Brute: ";
    for (int i = 0; i <= n - m; i++) {
```

```
        int j = 0;
        while (j < m && t[i + j] == p[j]) j++;
        if (j == m) cout << i << " ";
    }
    cout << "\n";
}

void kmp(char t[], char p[]) {
    int n = strlen(t), m = strlen(p), lps[30];
    if (m == 0 || n < m) return;
    for (int k = 0; k < m; k++) lps[k] = 0;
    int len = 0, i = 1;
    while (i < m) {
        if (p[i] == p[len]) lps[i++] = ++len;
        else if (len != 0) len = lps[len - 1];
        else lps[i++] = 0;
    }
    cout << "KMP  : ";
    i = 0; int j = 0;
    while (i < n) {
        if (p[j] == t[i]) { i++; j++; }
        if (j == m) {
            cout << (i - j) << " ";
            j = lps[j - 1];
        } else if (i < n && p[j] != t[i]) {
            j ? (j = lps[j - 1]) : i++;
        }
    }
    cout << "\n";
}

void main() {
    clrscr();
    char txt[] = "AABAACAADAABAABA", pat[] = "AABA";
    cout << "Text : " << txt << "\n";
    cout << "Pat  : " << pat << "\n";
    brute(txt, pat);
    kmp(txt, pat);
    getch();
}
```

OUTPUT

```
Text : AABAACAADAABAABA
Pat  : AABA
Brute: 0 9 12
KMP  : 0 9 12
```

Result: Pattern matching algorithms executed successfully in Turbo C++.

Program 13: Sorting Algorithms

AIM

To implement Bubble Sort, Selection Sort, and Insertion Sort in Turbo C++.

ALGORITHM (7 STEPS)

1. Start the program and initialize integer arrays for Bubble, Selection, and Insertion Sort.
2. In Bubble Sort: run outer loop i from 0 to n - 2; swap adjacent elements if a[j] > a[j + 1].
3. In Selection Sort: find index m of minimum element in unsorted range i to n - 1.
4. Swap minimum element a[m] with element a[i] at start of unsorted section.
5. In Insertion Sort: take element key = a[i] and shift larger preceding elements right.
6. Insert key into vacant position a[j + 1] = key.
7. Display sorted arrays for all three algorithms, and stop the program.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

void swap(int &x, int &y) {
    int t = x; x = y; y = t;
}

void bubble(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1])
                swap(a[j], a[j + 1]);
}
```

```
void selection(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int m = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[m]) m = j;
        swap(a[i], a[m]);
    }
}

void insertion(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int k = a[i], j = i - 1;
        while (j >= 0 && a[j] > k) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = k;
    }
}

void show(int a[], int n) {
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << "\n";
}

void main() {
    clrscr();
    int b[] = {4, 2, 1};
    int s[] = {5, 3, 2};
    int in[] = {9, 1, 6};
    bubble(b, 3);
    cout << "Bubble   : "; show(b, 3);
    selection(s, 3);
    cout << "Selection: "; show(s, 3);
    insertion(in, 3);
    cout << "Insertion: "; show(in, 3);
    getch();
}
```

OUTPUT

```
Bubble   : 1 2 4
Selection: 2 3 5
Insertion: 1 6 9
```
Result: Sorting algorithms executed successfully in Turbo C++.

Result: Sorting algorithms executed successfully in Turbo C++.

Program 14: Searching Algorithms

AIM

To implement Linear Search and Binary Search in Turbo C++.

ALGORITHM (7 STEPS)

1. Start the program and initialize a sorted integer array a[] of size n and search keys.
2. In Linear Search: scan array sequentially from index 0 to n - 1.
3. If a[i] == k, return index i; if loop completes without match, return -1.
4. In Binary Search: initialize search bounds low = 0 and high = n - 1.
5. Loop while low <= high: calculate midpoint m = low + (high - low) / 2.
6. If a[m] == k, return m; else if k < a[m], search left (high = m - 1); else search right (low = m + 1).
7. If low > high, return -1; print search results and stop the program.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

int linearSearch(int a[], int n, int k) {
    for (int i = 0; i < n; i++)
        if (a[i] == k) return i;
    return -1;
}

int binarySearch(int a[], int n, int k) {
    int l = 0, h = n - 1;
    while (l <= h) {
        int m = l + (h - l) / 2;
```

```
        if (a[m] == k) return m;
        if (a[m] < k) l = m + 1;
        else h = m - 1;
    }
    return -1;
}

void main() {
    clrscr();
    int a[] = {10, 20, 30, 40, 50}, n = 5;
    int l1 = linearSearch(a, n, 30);
    int b1 = binarySearch(a, n, 40);
    int b2 = binarySearch(a, n, 90);
    cout << "Linear (30): Idx " << l1 << "\n";
    cout << "Binary (40): Idx " << b1 << "\n";
    cout << "Binary (90): Idx " << b2 << "\n";
    getch();
}
```

OUTPUT

```
Linear (30): Idx 2
Binary (40): Idx 3
Binary (90): Idx -1
```

Result: Linear and Binary Search algorithms executed successfully in Turbo C++.

Program 15: Hash Table Implementation

AIM

To implement Hash Table collision handling via Linear Probing and Separate Chaining in Turbo C++.

ALGORITHM (7 STEPS)

1. Start the program, define table size S = 5, and hash function idx = key % S.
2. Initialize Linear Probing table lp[S] with -1 and Chaining table ch[S] with NULL.
3. In Linear Probing: compute idx = key % S; while slot is occupied and count < S, probe (idx + 1) % S.
4. If count < S, insert key at lp[idx]; else print table full to avoid infinite loop.
5. In Separate Chaining: allocate a new Node with key and link it to head of chain[idx].
6. Update ch[idx] to point to the new node.
7. Display both Linear Probing and Separate Chaining tables, and stop the program.

PROGRAM (TURBO C++)

```
#include <iostream.h>
#include <conio.h>

const int S = 5;

struct Node {
    int k;
    Node* next;
};

void linearProbe(int ht[], int key) {
    int idx = key % S, count = 0;
    while (ht[idx] != -1 && count < S) {
        idx = (idx + 1) % S;
        count++;
    }
    if (count < S) ht[idx] = key;
    else cout << "Table full for " << key << "\n";
}

void chain(Node* table[], int key) {
    int idx = key % S;
    Node* n = new Node;
    n->k = key;
    n->next = table[idx];
    table[idx] = n;
}

void main() {
    clrscr();
    int lp[S];
    Node* ch[S];
    for (int i = 0; i < S; i++) {
        lp[i] = -1;
        ch[i] = NULL;
    }
```

```
    linearProbe(lp, 5); linearProbe(lp, 10);
    chain(ch, 7); chain(ch, 12);
    cout << "Linear Probe:\n";
    for (int j = 0; j < S; j++) {
        cout << "[" << j << "] -> ";
        if (lp[j] == -1) cout << "Empty\n";
        else cout << lp[j] << "\n";
    }
    cout << "Chaining:\n";
    for (int m = 0; m < S; m++) {
        cout << "[" << m << "]: ";
        for (Node* t = ch[m]; t; t = t->next)
            cout << t->k << " -> ";
        cout << "NULL\n";
    }
    getch();
}
```

OUTPUT

```
Linear Probe:
[0] -> 5
[1] -> 10
[2] -> Empty
[3] -> Empty
[4] -> Empty
Chaining:
[0]: NULL
[1]: NULL
[2]: 12 -> 7 -> NULL
[3]: NULL
[4]: NULL
```

Result: Hash table collision resolution implemented and verified in Turbo C++.